Introduction to Deep Neural Networks3

Ang Qi Xuan Evan (S6865313) Mihai Bulubasa (s1234567)

October 19, 2026

Exercise 1 – Convolutional Autoencoder and Downstream Classification

Step 1 – Self-supervised pretraining(3 pts)

The autoencoder consists of a symmetric 4-layer encoder and 4-layer decoder architecture designed to process $32 \times 32 \times 3$ color images:

- **Input Normalization:** Raw CIFAR-10 images are normalized to the range $[-1, 1]$ using a channel mean of 0.5 and a standard deviation of 0.5.
- **Noise Corruption:** Additive Gaussian noise ($\sigma = 0.2$) was added to input images during training, while preserving clean original images as ground-truth targets.
- **Encoder Architecture:** 4 convolutional layers with stride-2 downsampling (`Conv2d` $\rightarrow$ `BatchNorm2d` $\rightarrow$ `LeakyReLU(0.2)`). The spatial resolution scales down as follows: $32\times32 \rightarrow 16 \times 16 \rightarrow 8 \times 8 \rightarrow 4 \times 4 \rightarrow 2 \times 2$. The bottleneck features consist of 256 channels, yielding a compressed latent representation of shape $2 \times 2 \times 256$ (1024 features).
- **Decoder Architecture:** 4 transposed convolutional layers with stride-2 upsampling (`ConvTranspose2d` $\rightarrow$ `BatchNorm2d` $\rightarrow$ `LeakyReLU(0.2)`).
- **Output Layer:** Ends with a `Tanh()` activation function to strictly enforce outputs within the normalized $[-1, 1]$ range.
- **Optimization & Loss:** The network was trained for 50 epochs using the Adam optimizer ($\text{lr} = 10^{-3}$) and Mean Squared Error (`nn.MSELoss`) reconstruction loss.

The autoencoder demonstrated rapid and smooth convergence over 50 epochs. Key loss values logged across training include:

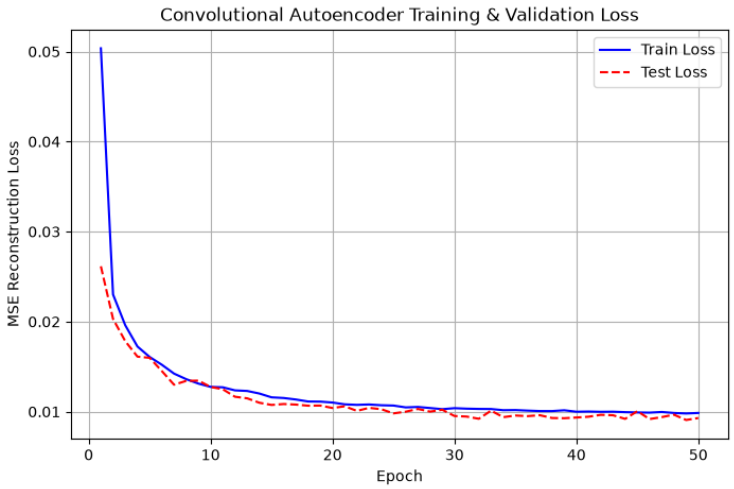
Table 1: Training and Test Reconstruction Loss (MSE) over 50 Epochs

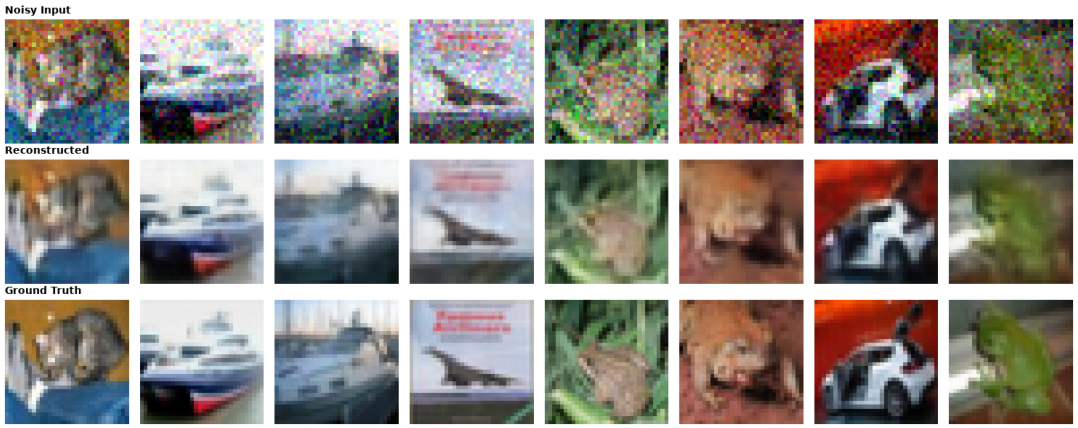
Epoch	Train Loss (MSE)	Test Loss (MSE)
1	0.05048	0.02601
10	0.01278	0.01264
20	0.01104	0.01053
30	0.01046	0.00958
40	0.01004	0.00938
50	0.00991	**0.00931**

Loss Curve Observations:

- **Rapid Convergence:** The reconstruction loss dropped significantly during the first 5 epochs from ~ 0.050 down to ~ 0.016.
- **Generalization:** Validation (test) loss aligned closely with training loss throughout all 50 epochs, settling around 0.0093 MSE without signs of overfitting.

Visual inspection of reconstructed images confirms that the model successfully strips heavy additive Gaussian noise across diverse object classes (e.g., ships, airplanes, frogs, automobiles). Primary global structures, colors, and background boundaries are well preserved. However, high-frequency fine details (such as facial features or fine text) exhibit minor blurring due to spatial compression into the 2×2 spatial bottleneck.





Step 2 – Downstream Classification (2 pts)

For downstream classification, the decoder was discarded and replaced by a classification head composed of Global Average Pooling (`AdaptiveAvgPool2d((1, 1))`) and a fully connected linear layer mapping 256 channels to 10 output class logits.

Two models were trained under identical conditions (30 epochs, Adam optimizer, lr $= 10^{-3}$, batch size 128, CrossEntropyLoss):

1. **Pretrained Encoder (Fine-tuned):** Encoder weights initialized from Task 1's saved autoencoder checkpoint (`conv_autoencoder_cifar10.pth`).

2. **Trained From Scratch:** Encoder weights initialized randomly.

Table 2: Downstream CIFAR-10 Classification Performance Comparison

	Pretrained Fine-Tuned		Trained From Scratch	
Epoch	**Train Acc (%)**	**Test Acc (%)**	**Train Acc (%)**	**Test Acc (%)**
10	83.35	**69.09**	84.48	**70.11**
20	94.35	68.44	95.36	69.28
30	97.30	69.06	98.03	68.56

1. **Test Accuracy Convergence:** Both initialization approaches reach virtually identical final test accuracy bounds ($\sim 68.5\% - 69.1\%$). Supervised gradient signals on a small model architecture prove sufficiently effective at finding a good local minimum without requiring pretraining.

2. **Overfitting Behavior:** Both networks exhibit strong overfitting after epoch 10, with training accuracy exceeding 97% while test accuracy plateaus near 69%. This highlights the need for explicit regularizers (e.g., Dropout or random crop/flip data augmentations) for further performance gains.

3. **Pretraining Limitations:** Unsupervised reconstruction forces the encoder to capture background elements and pixel-level L2 metrics, which do not strictly align with learning semantic class-discriminative features. Additionally, compressing images to a 2×2 latent bottleneck discards spatial fine details useful for class discrimination.

Exercise 2 – Generative adversarial network

We implemented a DCGAN-style [2] generative adversarial network [1] for the `CIFAR-10` training set (50 000 images of size $32 \times 32 \times 3$). Every image is converted with `ToTensor()` and then `Normalize((0.5,0.5,0.5),(0.5,0.5,0.5))`, which maps pixel values from $[0, 1]$ to $[-1, 1]$. This range sets the output activation of the generator (Step 1). The networks are defined in `gan.py` and the training script is `gan_train.py` (see Appendix A).

Step 1 – Generator (2pt)

The generator G maps a noise vector $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, $\mathbf{z} \in \mathbb{R}^{100}$, to an RGB image of size 32×32. The vector is reshaped to $100 \times 1 \times 1$ and passed through **four** transposed convolutional layers. The first layer projects it to a 4×4 feature map, and each later layer doubles the spatial resolution while halving the number of channels (Table 3). Every hidden layer is followed by batch normalisation and ReLU, which stabilises training and keeps the gradients flowing.

The last layer uses tanh, which outputs values in $[-1, 1]$. This matches the $[-1, 1]$ normalisation of the real images, so the discriminator cannot separate real from fake images by their value range alone.

Table 3: Generator architecture. ConvT(k, s, p) is a transposed convolution with kernel size k, stride s and padding p.

Layer	Operation	Activation	Output shape	Parameters
Input	reshape $\mathbf{z}$	–	$100 \times 1 \times 1$	0
1	ConvT$(4, 1, 0)$ + BN	ReLU	$256 \times 4 \times 4$	410 368
2	ConvT$(4, 2, 1)$ + BN	ReLU	$128 \times 8 \times 8$	524 672
3	ConvT$(4, 2, 1)$ + BN	ReLU	$64 \times 16 \times 16$	131 264
4	ConvT$(4, 2, 1)$	tanh	$3 \times 32 \times 32$	3 075
Total				1 069 379

Listing 1: Generator (`gan.py`).

```python
class Generator(nn.Module):
    def __init__(self, noise_dim):
        super(Generator, self).__init__()
        self.noise_dim = noise_dim
        self.main = nn.Sequential(
            # L1
            nn.ConvTranspose2d(noise_dim, 256, kernel_size=4, stride=1),
            nn.BatchNorm2d(256),
            nn.ReLU(True),
            # L2
            nn.ConvTranspose2d(256, 128, kernel_size=4, stride=2, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(True),
            # L3
            nn.ConvTranspose2d(128, 64, kernel_size=4, stride=2, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(True),
            # L4
            nn.ConvTranspose2d(64, 3, kernel_size=4, stride=2, padding=1),
            nn.Tanh()
        )

    def forward(self, x):
        x = x.view(-1, self.noise_dim, 1, 1)
        return self.main(x)
```

Step 2 – Discriminator (2pt)

The discriminator D takes a real or generated $3 \times 32 \times 32$ image and outputs a single real/fake score. It mirrors the generator and has **four** convolutional layers (Table 4). The first three are strided convolutions ($k = 4$, $s = 2$, $p = 1$) that halve the spatial resolution ($32 \to 16 \to 8 \to 4$) and use strided convolution in place of pooling for downsampling. These layers use LeakyReLU with slope 0.2, which keeps a non-zero gradient for negative inputs, so the generator still receives useful gradients when the discriminator is confident. Batch normalisation follows each of them.

Classification head. The fourth layer is a 4×4 convolution with a single output channel and no padding. It reduces the $256 \times 4 \times 4$ feature map to one value per image, so it acts as a fully connected layer over the final features. This value is a *logit*. We apply no sigmoid in the network and instead use `nn.BCEWithLogitsLoss`, which combines the sigmoid $\sigma(\cdot)$ and the binary cross-entropy in one numerically stable operation. The probability that an image is real is therefore $D(\mathbf{x}) = \sigma(\text{logit})$.

Table 4: Discriminator architecture. Conv(k, s, p) is a convolution with kernel size k, stride s and padding p.

Layer	Operation	Activation	Output shape	Parameters
Input	image	–	$3 \times 32 \times 32$	0
1	Conv$(4, 2, 1)$ + BN	LeakyReLU(0.2)	$64 \times 16 \times 16$	3 264
2	Conv$(4, 2, 1)$ + BN	LeakyReLU(0.2)	$128 \times 8 \times 8$	131 328
3	Conv$(4, 2, 1)$ + BN	LeakyReLU(0.2)	$256 \times 4 \times 4$	524 800
4 (head)	Conv$(4, 1, 0)$	– (logit)	$1 \times 1 \times 1$	4 097
Total				663 489

Listing 2: Discriminator (`gan.py`).

```python
class Discriminator(nn.Module):
    def __init__(self):
        super(Discriminator, self).__init__()
        self.main = nn.Sequential(
            # L1
            nn.Conv2d(3, 64, kernel_size=4, stride=2, padding=1),
            nn.BatchNorm2d(64),
            nn.LeakyReLU(0.2, inplace=True),

            # L2
            nn.Conv2d(64, 128, kernel_size=4, stride=2, padding=1, bias=False),
            nn.BatchNorm2d(128),
            nn.LeakyReLU(0.2, inplace=True),
            # L3
            nn.Conv2d(128, 256, kernel_size=4, stride=2, padding=1, bias=False),
            nn.BatchNorm2d(256),
            nn.LeakyReLU(0.2, inplace=True),
            # L4
            nn.Conv2d(256, 1, kernel_size=4, stride=1)
        )

    def forward(self, x):
        return self.main(x)
```

Step 3 – Training the GAN (1pt)

Objective. G and D play the minimax game

$$\min_G \max_D \; V(D,G) = \mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}\left[\log D(\mathbf{x})\right] + \mathbb{E}_{\mathbf{z}\sim p_{\mathbf{z}}}\left[\log\left(1 - D(G(\mathbf{z}))\right)\right]. \tag{1}$$

In practice, both players minimise a binary cross-entropy (BCE) loss. The discriminator labels real images 1 and generated images 0:

$$\mathcal{L}_D = -\frac{1}{m}\sum_{i=1}^{m}\left[\log D(\mathbf{x}_i) + \log\left(1 - D(G(\mathbf{z}_i))\right)\right]. \tag{2}$$

For the generator we use the *non-saturating* loss. It trains G to make D label its samples as real (target 1) and gives stronger gradients early in training than minimising $\log(1 - D(G(\mathbf{z})))$ directly:

$$\mathcal{L}_G = -\frac{1}{m}\sum_{i=1}^{m}\log D(G(\mathbf{z}_i)). \tag{3}$$

Procedure. Each mini-batch has two alternating updates. (i) D **step:** compute the BCE on a batch of real images (target 1) and on a batch of generated images (target 0). The generated images are `detach()`-ed so that no gradient flows into G. Then update D. (ii) G **step:** pass the same generated images through the updated D with target 1 and update G only. Table 5 lists the hyperparameters, which follow the DCGAN recommendations [2]. After epochs 1, 5, 10, 20, …, 100 we generate images from a *fixed* set of 16 noise vectors, so the progress of the generator can be compared on the same inputs.

Table 5: Training hyperparameters.

Setting	Value
Loss	`BCEWithLogitsLoss`
Optimiser (G and D)	Adam, lr $= 2 \times 10^{-4}$, $\beta_1 = 0.5$, $\beta_2 = 0.999$
Batch size	256
Epochs	100
Noise dimension	100, $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
Data augmentation	none
Hardware	Hábrók cluster, 1 GPU

Listing 3: Adversarial training loop (`gan_train.py`).

```
for epoch in range(NUM_EPOCHS):
    dis_running, gen_running = 0.0, 0.0
    for i, data in enumerate(trainloader):
        real_images, _ = data
        real_images = real_images.to(device)

        dis_optimiser.zero_grad()
        real_labels = torch.ones(real_images.size(0), 1, 1, 1, device=device)
        real_outputs = dis(real_images)
        real_loss = criterion(real_outputs, real_labels)
        real_loss.backward()

        noise = torch.randn(real_images.size(0), NOISE_DIM, device=device)
        fake_images = gen(noise)
        fake_labels = torch.zeros(real_images.size(0), 1, 1, 1, device=device)
        fake_outputs = dis(fake_images.detach())
        fake_loss = criterion(fake_outputs, fake_labels)
        fake_loss.backward()
        dis_optimiser.step()

        gen_optimiser.zero_grad()
        fake_labels = torch.ones(real_images.size(0), 1, 1, 1, device=device)
        fake_outputs = dis(fake_images)
        gen_loss = criterion(fake_outputs, fake_labels)
        gen_loss.backward()
        gen_optimiser.step()

        dis_running += real_loss.item() + fake_loss.item()
        gen_running += gen_loss.item()

    dis_epoch = dis_running / len(trainloader)
    gen_epoch = gen_running / len(trainloader)
    dis_losses.append(dis_epoch)
    gen_losses.append(gen_epoch)
    print(f'Epoch {epoch+1}: D loss {dis_epoch:.4f}, G loss {gen_epoch:.4f}')

    if epoch + 1 in (1, 5) or (epoch + 1) % 10 == 0:
        generate_and_save_images(gen, epoch, fixed_noise)
```

```
        gen.train()
```

Loss curves. Figure 1 shows the average loss per epoch. The reported $\mathcal{L}_D$ is the sum of the real and fake BCE terms.

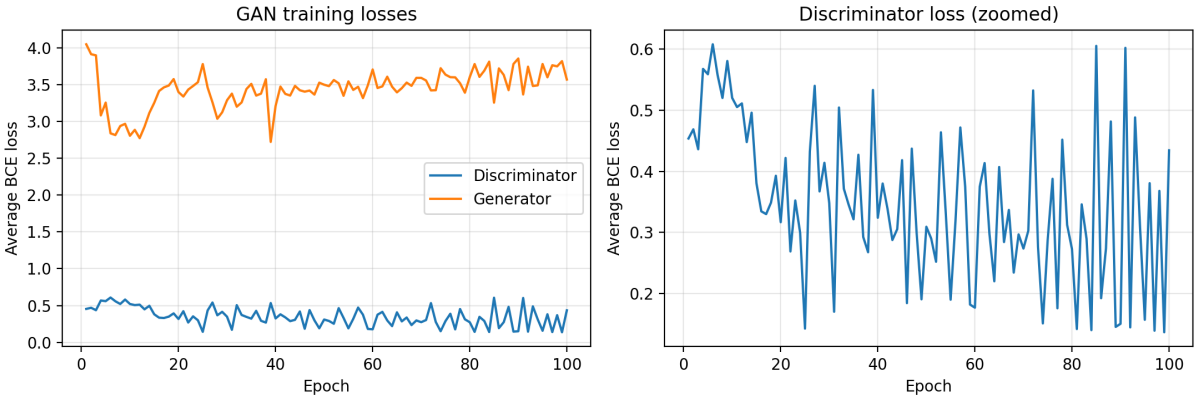


Figure 1: Generator and discriminator BCE loss per epoch (left) and a zoomed view of the discriminator loss (right).

Table 6: Average losses at selected epochs.

Epoch	$\mathcal{L}_D$	$\mathcal{L}_G$
1	0.454	4.050
5	0.559	3.257
10	0.520	2.807
25	0.142	3.781
50	0.309	3.501
75	0.288	3.638
100	0.434	3.571

The losses do not converge to a single value, which is expected: G and D optimise opposing objectives, so a low loss for one player means a high loss for the other. At the theoretical equilibrium $D(\cdot) = 0.5$ everywhere, which would give $\mathcal{L}_D = 2\ln 2 \approx 1.39$ and $\mathcal{L}_G = \ln 2 \approx 0.69$. In our run, $\mathcal{L}_D$ stays between about 0.15 and 0.6, and $\mathcal{L}_G$ drops to about 2.8 around epoch 10 and then settles around 3.5. This corresponds to $D(G(\mathbf{z})) \approx e^{-3.5} \approx 0.03$ on average. The discriminator therefore stays ahead of the generator for most of training. The discriminator loss oscillates strongly from epoch to epoch, but neither loss diverges, so training does not collapse.

Generated images. Figure 2 shows images generated from the same fixed noise vectors at different epochs, and Figure 3 shows 64 random samples from the final generator.



Figure 2: Images generated from the same 16 fixed noise vectors after epochs 1 to 100.

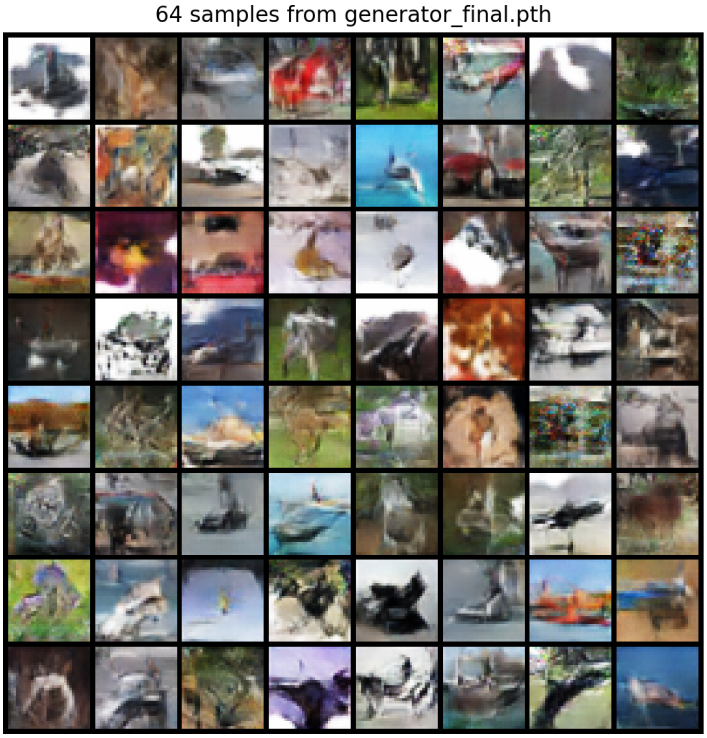


Figure 3: 64 random samples from the generator after 100 epochs.

After epoch 1 the samples are colour blobs with a strong checkerboard pattern, which is typical of transposed convolutions early in training. By epoch 5 to 10 the generator has learned global colour layouts such as sky or ground, background versus foreground, and rough object silhouettes. From about epoch 30 onwards, each noise vector maps to a stable composition, and later epochs mainly sharpen edges and textures. The final samples are diverse in colour and layout, and several resemble CIFAR-10 classes such as cars, ships, horses and birds. We see no sign of mode collapse. Most objects are still not clearly recognisable, and a few samples still show high-frequency, noisy artefacts.

References

[1] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in*

Neural Information Processing Systems, volume 27, 2014.

[2] Alec Radford, Luke Metz, and Soumith Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. *arXiv preprint arXiv:1511.06434*, 2015.

A Exercise 2 code

Listing 4: Generator and discriminator definitions, `gan.py`.

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import numpy as np

class Generator(nn.Module):
    def __init__(self, noise_dim):
        super(Generator, self).__init__()
        self.noise_dim = noise_dim
        self.main = nn.Sequential(
            # L1
            nn.ConvTranspose2d(noise_dim, 256, kernel_size=4, stride=1),
            nn.BatchNorm2d(256),
            nn.ReLU(True),
            # L2
            nn.ConvTranspose2d(256, 128, kernel_size=4, stride=2, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(True),
            # L3
            nn.ConvTranspose2d(128, 64, kernel_size=4, stride=2, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(True),
            # L4
            nn.ConvTranspose2d(64, 3, kernel_size=4, stride=2, padding=1),
            nn.Tanh()
        )

    def forward(self, x):
        x = x.view(-1, self.noise_dim, 1, 1)
        return self.main(x)

class Discriminator(nn.Module):
    def __init__(self):
        super(Discriminator, self).__init__()
        self.main = nn.Sequential(
            # L1
            nn.Conv2d(3, 64, kernel_size=4, stride=2, padding=1),
            nn.BatchNorm2d(64),
            nn.LeakyReLU(0.2, inplace=True),

            # L2
            nn.Conv2d(64, 128, kernel_size=4, stride=2, padding=1, bias=False),
            nn.BatchNorm2d(128),
            nn.LeakyReLU(0.2, inplace=True),
            # L3
            nn.Conv2d(128, 256, kernel_size=4, stride=2, padding=1, bias=False),
            nn.BatchNorm2d(256),
            nn.LeakyReLU(0.2, inplace=True),
            # L4
            nn.Conv2d(256, 1, kernel_size=4, stride=1)
        )

    def forward(self, x):
        return self.main(x)
```

Listing 5: GAN training script, `gan_train.py`.

```
import torch
import torch.nn as nn
import torch.optim as optim

import torchvision
import torchvision.transforms as transforms
from torchvision.utils import save_image, make_grid

import os
import argparse

from gan import Generator, Discriminator

parser = argparse.ArgumentParser(description='PyTorch CIFAR10 GAN Training')
parser.add_argument('--lr', default=0.0002, type=float, help='learning rate')
parser.add_argument('--ep', default=50, type=int, help='num of epochs')
args = parser.parse_args()

NOISE_DIM = 100

gen = Generator(NOISE_DIM)
dis = Discriminator()

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
gen.to(device)
dis.to(device)

criterion = nn.BCEWithLogitsLoss()
gen_optimiser = optim.Adam(gen.parameters(), lr = args.lr, betas=(0.5, 0.999))
dis_optimiser = optim.Adam(dis.parameters(), lr = args.lr, betas=(0.5, 0.999))

NUM_EPOCHS = args.ep
BATCH_SIZE = 256

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,0.5,0.5),(0.5,0.5,0.5))
])

trainset = torchvision.datasets.CIFAR10(
    root='./data', train=True, download=True, transform=transform)
trainloader = torch.utils.data.DataLoader(
    trainset, batch_size=BATCH_SIZE, shuffle=True, num_workers=2)

def generate_and_save_images(model, epoch, noise):
    model.eval()
    with torch.no_grad():
        fake_images = model(noise).cpu()

    save_image(fake_images, f'test_images/image_at_epoch_{epoch+1:04d}.png',
               nrow=4, normalize=True, value_range=(-1, 1))

dis_losses, gen_losses = [], []
fixed_noise = torch.randn(16, NOISE_DIM, device=device)

for epoch in range(NUM_EPOCHS):
    dis_running, gen_running = 0.0, 0.0
    for i, data in enumerate(trainloader):
        real_images, _ = data
        real_images = real_images.to(device)

        dis_optimiser.zero_grad()
        real_labels = torch.ones(real_images.size(0), 1, 1, 1, device=device)
```

```
        real_outputs = dis(real_images)
        real_loss = criterion(real_outputs, real_labels)
        real_loss.backward()

        noise = torch.randn(real_images.size(0), NOISE_DIM, device=device)
        fake_images = gen(noise)
        fake_labels = torch.zeros(real_images.size(0), 1, 1, 1, device=device)
        fake_outputs = dis(fake_images.detach())
        fake_loss = criterion(fake_outputs, fake_labels)
        fake_loss.backward()
        dis_optimiser.step()

        gen_optimiser.zero_grad()
        fake_labels = torch.ones(real_images.size(0), 1, 1, 1, device=device)
        fake_outputs = dis(fake_images)
        gen_loss = criterion(fake_outputs, fake_labels)
        gen_loss.backward()
        gen_optimiser.step()

        dis_running += real_loss.item() + fake_loss.item()
        gen_running += gen_loss.item()

    dis_epoch = dis_running / len(trainloader)
    gen_epoch = gen_running / len(trainloader)
    dis_losses.append(dis_epoch)
    gen_losses.append(gen_epoch)
    print(f'Epoch {epoch+1}: D loss {dis_epoch:.4f}, G loss {gen_epoch:.4f}')

    if epoch + 1 in (1, 5) or (epoch + 1) % 10 == 0:
        generate_and_save_images(gen, epoch, fixed_noise)
        gen.train()

torch.save(dis.state_dict(), 'discriminator_final.pth')
torch.save(gen.state_dict(), 'generator_final.pth')

print(dis_losses)
print(gen_losses)
```